

TP Final ISSE - Estación Meteorológica

Generated by Doxygen 1.18.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 Core Directory Reference	7
4.2 Core/Inc Directory Reference	7
4.3 Core/Src Directory Reference	8
5 Class Documentation	9
5.1 AHT20Sensor Struct Reference	9
5.1.1 Detailed Description	9
5.1.2 Member Data Documentation	9
5.1.2.1 address_	9
5.1.2.2 calibrated_	10
5.1.2.3 hi2c_	10
5.1.2.4 humidity_	10
5.1.2.5 temperature_	10
5.2 SerialDisplay Struct Reference	10
5.2.1 Detailed Description	10
5.2.2 Member Data Documentation	10
5.2.2.1 huart_	10
6 File Documentation	11
6.1 Core/Inc/aht20.h File Reference	11
6.1.1 Detailed Description	12
6.1.2 Macro Definition Documentation	12
6.1.2.1 AHT20_I2C_ADDRESS	12
6.1.3 Function Documentation	12
6.1.3.1 AHT20_GetHumidity()	12
6.1.3.2 AHT20_GetTemperature()	12
6.1.3.3 AHT20_Init()	13
6.1.3.4 AHT20_ReadMeasurement()	13
6.2 aht20.h	14
6.3 Core/Inc/main.h File Reference	15
6.3.1 Detailed Description	15
6.3.2 Function Documentation	15
6.3.2.1 Error_Handler()	15
6.4 main.h	16

6.5 Core/Inc/serial_display.h File Reference	16
6.5.1 Detailed Description	17
6.5.2 Function Documentation	17
6.5.2.1 SerialDisplay_Init()	17
6.5.2.2 SerialDisplay_PrintError()	18
6.5.2.3 SerialDisplay_PrintMeasurement()	18
6.6 serial_display.h	19
6.7 Core/Inc/timer.h File Reference	19
6.7.1 Detailed Description	20
6.7.2 Function Documentation	20
6.7.2.1 Timer_Delay()	20
6.7.2.2 Timer_Millis()	20
6.8 timer.h	21
6.9 Core/Src/aht20.c File Reference	21
6.9.1 Detailed Description	21
6.9.2 Macro Definition Documentation	22
6.9.2.1 AHT20_CMD_INIT	22
6.9.2.2 AHT20_CMD_MEASURE	22
6.9.2.3 AHT20_I2C_TIMEOUT_MS	22
6.9.2.4 AHT20_STATUS_BUSY_MASK	22
6.9.2.5 AHT20_STATUS_CAL_MASK	22
6.9.3 Function Documentation	22
6.9.3.1 AHT20_GetHumidity()	22
6.9.3.2 AHT20_GetTemperature()	23
6.9.3.3 AHT20_Init()	23
6.9.3.4 AHT20_ReadMeasurement()	24
6.9.3.5 AHT20_ReadStatus()	24
6.10 aht20.c	25
6.11 Core/Src/main.c File Reference	26
6.11.1 Detailed Description	26
6.11.2 Function Documentation	27
6.11.2.1 Error_Handler()	27
6.11.2.2 main()	27
6.11.2.3 MX_GPIO_Init()	28
6.11.2.4 MX_I2C1_Init()	28
6.11.2.5 MX_USART2_UART_Init()	29
6.11.2.6 SystemClock_Config()	30
6.11.3 Variable Documentation	31
6.11.3.1 hi2c1	31
6.11.3.2 huart2	31
6.12 main.c	31
6.13 Core/Src/serial_display.c File Reference	33

6.13.1 Detailed Description	34
6.13.2 Macro Definition Documentation	34
6.13.2.1 SERIAL_TX_TIMEOUT_MS	34
6.13.3 Function Documentation	35
6.13.3.1 SerialDisplay_Init()	35
6.13.3.2 SerialDisplay_PrintError()	35
6.13.3.3 SerialDisplay_PrintMeasurement()	36
6.14 serial_display.c	36
6.15 Core/Src/timer.c File Reference	37
6.15.1 Detailed Description	37
6.15.2 Function Documentation	37
6.15.2.1 Timer_Delay()	37
6.15.2.2 Timer_Millis()	38
6.16 timer.c	38

Chapter 1

Directory Hierarchy

1.1 Directories

Core	7
Inc	7
aht20.h	11
app.h	??
main.h	15
serial_display.h	16
timer.h	19
Src	8
aht20.c	21
app.c	??
main.c	26
serial_display.c	33
timer.c	37
Inc	7
aht20.h	11
app.h	??
main.h	15
serial_display.h	16
timer.h	19
Src	8
aht20.c	21
app.c	??
main.c	26
serial_display.c	33
timer.c	37

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AHT20Sensor	Estructura "publica" del objeto AHT20Sensor	9
App	Definición real de App . Es privada a app.c : fuera de este archivo solo se conoce como "struct App" (tipo incompleto), por lo que main.c no puede acceder a estos campos ni le importa como estan organizados (encapsulamiento)	??
SerialDisplay		10

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Core/Inc/ aht20.h	Driver "orientado a objetos" (en C) para el sensor AHT20 (temperatura + humedad, bus I2C) .	11
Core/Inc/ app.h	"Clase" App : contiene TODA la lógica del programa (maquina de estados del super-loop, orquestación de AHT20Sensor , Timer y SerialDisplay). Es el único modulo del TP que main.c conoce	??
Core/Inc/ main.h	: Header for main.c file. This file contains the common defines of the application	15
Core/Inc/ serial_display.h	"Clase" SerialDisplay : envía texto por el UART2 (puerto virtual del ST-LINK) para mostrar la temperatura en una terminal PuTTY, a 115200 8N1	16
Core/Inc/ timer.h	Modulo "utility" con métodos estáticos para medir tiempo sin bloquear el super-loop (RNF-01) y sin acoplar el resto del software a una implementacion concreta de reloj (RNF-03)	19
Core/Src/ aht20.c	Implementación del driver AHT20 (I2C)	21
Core/Src/ app.c	Implementación de la clase App . Aca vive toda la lógica del programa: la definición real de la estructura (privada a este archivo, siguiendo el patron de "puntero a estructura anonima" visto en clase 7 para representar objetos en C) y la máquina de estados del super-loop	??
Core/Src/ main.c	: Main program body	26
Core/Src/ serial_display.c	Implementación de la interfaz de salida por UART para mostrar mediciones y errores	33
Core/Src/ timer.c	Implementación de las funciones de temporización del sistema	37

Chapter 4

Directory Documentation

4.1 Core Directory Reference

Directories

- directory [Inc](#)
- directory [Src](#)

4.2 Core/Inc Directory Reference

Files

- file [aht20.h](#)
Driver "orientado a objetos" (en C) para el sensor AHT20 (temperatura + humedad, bus I2C).
- file [app.h](#)
"Clase" [App](#): contiene TODA la lógica del programa (maquina de estados del super-loop, orquestación de [AHT20Sensor](#), [Timer](#) y [SerialDisplay](#)). Es el único modulo del TP que [main.c](#) conoce.
- file [main.h](#)
: Header for [main.c](#) file. This file contains the common defines of the application.
- file [serial_display.h](#)
"Clase" [SerialDisplay](#): envía texto por el UART2 (puerto virtual del ST-LINK) para mostrar la temperatura en una terminal PuTTY, a 115200 8N1.
- file [timer.h](#)
Modulo "utility" con métodos estáticos para medir tiempo sin bloquear el super-loop (RNF-01) y sin acoplar el resto del software a una implementacion concreta de reloj (RNF-03).

4.3 Core/Src Directory Reference

Files

- file [aht20.c](#)
Implementación del driver AHT20 (I2C).
- file [app.c](#)
Implementación de la clase [App](#). Aca vive toda la lógica del programa: la definición real de la estructura (privada a este archivo, siguiendo el patron de "puntero a estructura anonima" visto en clase 7 para representar objetos en C) y la máquina de estados del super-loop.
- file [main.c](#)
: Main program body
- file [serial_display.c](#)
Implementación de la interfaz de salida por UART para mostrar mediciones y errores.
- file [timer.c](#)
Implementación de las funciones de temporización del sistema.

Chapter 5

Class Documentation

5.1 AHT20Sensor Struct Reference

Estructura "publica" del objeto [AHT20Sensor](#).

```
#include <aht20.h>
```

Public Attributes

- I2C_HandleTypeDef * [hi2c_](#)
- uint8_t [address_](#)
- float [temperature_](#)
- float [humidity_](#)
- bool [calibrated_](#)

5.1.1 Detailed Description

Estructura "publica" del objeto [AHT20Sensor](#).

Los campos con sufijo `_` son privados por convencion (RNF-04) y no deben modificarse fuera de [aht20.c](#); se acceden mediante los metodos [AHT20_GetTemperature\(\)](#) / [AHT20_GetHumidity\(\)](#).

Definition at line [33](#) of file [aht20.h](#).

5.1.2 Member Data Documentation

5.1.2.1 `address_`

```
uint8_t AHT20Sensor::address_
```

Definition at line [35](#) of file [aht20.h](#).

5.1.2.2 calibrated_

```
bool AHT20Sensor::calibrated_
```

Definition at line 38 of file [aht20.h](#).

5.1.2.3 hi2c_

```
I2C_HandleTypeDef* AHT20Sensor::hi2c_
```

Definition at line 34 of file [aht20.h](#).

5.1.2.4 humidity_

```
float AHT20Sensor::humidity_
```

Definition at line 37 of file [aht20.h](#).

5.1.2.5 temperature_

```
float AHT20Sensor::temperature_
```

Definition at line 36 of file [aht20.h](#).

The documentation for this struct was generated from the following file:

- Core/Inc/[aht20.h](#)

5.2 App Struct Reference

Definición real de [App](#). Es privada a [app.c](#): fuera de este archivo solo se conoce como "struct App" (tipo incompleto), por lo que [main.c](#) no puede acceder a estos campos ni le importa como estan organizados (encapsulamiento).

Public Attributes

- [AHT20Sensor sensor_](#)
- [SerialDisplay display_](#)
- [AppState state_](#)
- [uint32_t lastReadTime_](#)
- [uint32_t errorTime_](#)

5.2.1 Detailed Description

Definición real de [App](#). Es privada a [app.c](#): fuera de este archivo solo se conoce como "struct App" (tipo incompleto), por lo que [main.c](#) no puede acceder a estos campos ni le importa como estan organizados (encapsulamiento).

Definition at line 38 of file [app.c](#).

5.2.2 Member Data Documentation

5.2.2.1 display_

`SerialDisplay` `App::display_`

Definition at line 40 of file [app.c](#).

5.2.2.2 errorTime_

`uint32_t` `App::errorTime_`

Definition at line 43 of file [app.c](#).

5.2.2.3 lastReadTime_

`uint32_t` `App::lastReadTime_`

Definition at line 42 of file [app.c](#).

5.2.2.4 sensor_

`AHT20Sensor` `App::sensor_`

Definition at line 39 of file [app.c](#).

5.2.2.5 state_

`AppState` `App::state_`

Definition at line 41 of file [app.c](#).

The documentation for this struct was generated from the following file:

- [Core/Src/app.c](#)

5.3 SerialDisplay Struct Reference

```
#include <serial_display.h>
```

Public Attributes

- `UART_HandleTypeDef` * [huart_](#)

5.3.1 Detailed Description

Definition at line 20 of file [serial_display.h](#).

5.3.2 Member Data Documentation

5.3.2.1 huart_

`UART_HandleTypeDef*` `SerialDisplay::huart_`

Definition at line 21 of file [serial_display.h](#).

The documentation for this struct was generated from the following file:

- [Core/Inc/serial_display.h](#)

Chapter 6

File Documentation

6.1 Core/Inc/aht20.h File Reference

Driver "orientado a objetos" (en C) para el sensor AHT20 (temperatura + humedad, bus I2C).

```
#include <stdint.h>
#include <stdbool.h>
#include "stm32f4xx_hal.h"
```

Classes

- struct [AHT20Sensor](#)
Estructura "publica" del objeto [AHT20Sensor](#).

Macros

- #define [AHT20_I2C_ADDRESS](#) (0x38U << 1)

Functions

- bool [AHT20_Init](#) ([AHT20Sensor](#) *self, I2C_HandleTypeDef *hi2c)
Inicializa el objeto sensor y verifica/calibra el AHT20.
- bool [AHT20_ReadMeasurement](#) ([AHT20Sensor](#) *self)
Dispara una medición y actualiza temperature_/humidity_. Es una operación semi-bloqueante (~80 ms) por el tiempo de conversión propio del sensor; se llama desde el estado MEASURING del super-loop, no dentro de un ISR.
- float [AHT20_GetTemperature](#) (const [AHT20Sensor](#) *self)
Última temperatura valida leida, en grados Celsius.
- float [AHT20_GetHumidity](#) (const [AHT20Sensor](#) *self)
Última humedad relativa valida leida, en RH.

6.1.1 Detailed Description

Driver "orientado a objetos" (en C) para el sensor AHT20 (temperatura + humedad, bus I2C).

Implementa el patron visto en clase: cada objeto es un puntero a una estructura opaca (handle). Los atributos son privados al .c y solo se accede a ellos a traves de los metodos publicos declarados en [aht20.h](#).

Definition in file [aht20.h](#).

6.1.2 Macro Definition Documentation

6.1.2.1 AHT20_I2C_ADDRESS

```
#define AHT20_I2C_ADDRESS (0x38U << 1)
```

Definition at line 24 of file [aht20.h](#).

6.1.3 Function Documentation

6.1.3.1 AHT20_GetHumidity()

```
float AHT20_GetHumidity (  
    const AHT20Sensor * self)
```

Última humedad relativa valida leida, en RH.

Definition at line 100 of file [aht20.c](#).

```
00101 {  
00102     return self->humidity_  
00103 }
```

References [AHT20_GetHumidity\(\)](#), and [AHT20Sensor::humidity_](#).

6.1.3.2 AHT20_GetTemperature()

```
float AHT20_GetTemperature (  
    const AHT20Sensor * self)
```

Última temperatura valida leida, en grados Celsius.

Definition at line 95 of file [aht20.c](#).

```
00096 {  
00097     return self->temperature_  
00098 }
```

References [AHT20_GetTemperature\(\)](#), and [AHT20Sensor::temperature_](#).

6.1.3.3 AHT20_Init()

```
bool AHT20_Init (
    AHT20Sensor * self,
    I2C_HandleTypeDef * hi2c)
```

Inicializa el objeto sensor y verifica/calibra el AHT20.

Parameters

<i>self</i>	instancia del sensor (ya con hi2c_ asignado por el caller)
-------------	--

Return values

<i>true</i>	si el sensor respondió y quedó calibrado, false si no.
-------------	--

Definition at line 26 of file [ah20.c](#).

```
00027 {
00028     uint8_t init_cmd[3] = AHT20_CMD_INIT;
00029     uint8_t status = 0;
00030
00031     self->hi2c_ = hi2c;
00032     self->address_ = AHT20_I2C_ADDRESS;
00033     self->temperature_ = 0.0f;
00034     self->humidity_ = 0.0f;
00035     self->calibrated_ = false;
00036
00037     HAL_Delay(40); /* tiempo minimo de power-on del AHT20 */
00038
00039     if (!AHT20_ReadStatus(self, &status)) {
00040         return false;
00041     }
00042
00043     if ((status & AHT20_STATUS_CAL_MASK) == 0U) {
00044         if (HAL_I2C_Master_Transmit(self->hi2c_, self->address_, init_cmd,
00045                                     sizeof(init_cmd), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00046             return false;
00047         }
00048         HAL_Delay(10);
00049     }
00050
00051     if (!AHT20_ReadStatus(self, &status)) {
00052         return false;
00053     }
00054
00055     self->calibrated_ = ((status & AHT20_STATUS_CAL_MASK) != 0U);
00056     return self->calibrated_;
00057 }
```

References [AHT20Sensor::address_](#), [AHT20_CMD_INIT](#), [AHT20_I2C_ADDRESS](#), [AHT20_I2C_TIMEOUT_MS](#), [AHT20_Init\(\)](#), [AHT20_ReadStatus\(\)](#), [AHT20_STATUS_CAL_MASK](#), [AHT20Sensor::calibrated_](#), [AHT20Sensor::hi2c_](#), [AHT20Sensor::humidity_](#), [status](#), and [AHT20Sensor::temperature_](#).

6.1.3.4 AHT20_ReadMeasurement()

```
bool AHT20_ReadMeasurement (
    AHT20Sensor * self)
```

Dispara una medición y actualiza temperature_/humidity_. Es una operación semi-bloqueante (~80 ms) por el tiempo de conversión propio del sensor; se llama desde el estado MEASURING del super-loop, no dentro de un ISR.

Return values

<code>true</code>	si la lectura fue valida (ACK + CRC de estado ok).
-------------------	--

Definition at line 59 of file [aht20.c](#).

```

00060 {
00061     uint8_t measure_cmd[3] = AHT20_CMD_MEASURE;
00062     uint8_t data[6] = {0};
00063     uint32_t raw_hum;
00064     uint32_t raw_temp;
00065
00066     if (!self->calibrated_) {
00067         return false;
00068     }
00069
00070     if (HAL_I2C_Master_Transmit(self->hi2c_, self->address_, measure_cmd,
00071                                sizeof(measure_cmd), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00072         return false;
00073     }
00074
00075     HAL_Delay(80); /* tiempo de conversion tipico del AHT20 */
00076
00077     if (HAL_I2C_Master_Receive(self->hi2c_, self->address_, data,
00078                                sizeof(data), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00079         return false;
00080     }
00081
00082     if ((data[0] & AHT20_STATUS_BUSY_MASK) != 0U) {
00083         return false; /* el sensor seguia ocupado: descartar lectura */
00084     }
00085
00086     raw_hum = ((uint32_t)data[1] << 12) | ((uint32_t)data[2] << 4) | (data[3] >> 4);
00087     raw_temp = (((uint32_t)data[3] & 0x0FU) << 16) | ((uint32_t)data[4] << 8) | data[5];
00088
00089     self->humidity_ = ((float)raw_hum / 1048576.0f) * 100.0f;
00090     self->temperature_ = (((float)raw_temp / 1048576.0f) * 200.0f) - 50.0f;
00091
00092     return true;
00093 }

```

References [AHT20Sensor::address_](#), [AHT20_CMD_MEASURE](#), [AHT20_I2C_TIMEOUT_MS](#), [AHT20_ReadMeasurement\(\)](#), [AHT20_STATUS_BUSY_MASK](#), [AHT20Sensor::calibrated_](#), [AHT20Sensor::hi2c_](#), [AHT20Sensor::humidity_](#), and [AHT20Sensor::temperature_](#).

6.2 aht20.h

[Go to the documentation of this file.](#)

```

00001
00012 #ifndef AHT20_H
00013 #define AHT20_H
00014
00015 #ifdef __cplusplus
00016 extern "C" {
00017 #endif
00018
00019 #include <stdint.h>
00020 #include <stdbool.h>
00021 #include "stm32f4xx_hal.h"
00022
00023 /* Direccion I2C de 7 bits del AHT20 (fija por hardware) */
00024 #define AHT20_I2C_ADDRESS (0x38U << 1)
00025
00033 typedef struct {
00034     I2C_HandleTypeDef *hi2c_; /* bus I2C utilizado (HAL) */
00035     uint8_t address_; /* direccion I2C del dispositivo */
00036     float temperature_;
00037     float humidity_;
00038     bool calibrated_;
00039 } AHT20Sensor;
00040
00041 /* ---- Metodos publicos (API del "objeto" AHT20Sensor) ----- */
00042
00048 bool AHT20_Init(AHT20Sensor *self, I2C_HandleTypeDef *hi2c);
00049
00057 bool AHT20_ReadMeasurement(AHT20Sensor *self);

```

```

00058
00060 float AHT20_GetTemperature(const AHT20Sensor *self);
00061
00063 float AHT20_GetHumidity(const AHT20Sensor *self);
00064
00065 #ifdef __cplusplus
00066 }
00067 #endif
00068
00069 #endif /* AHT20_H */

```

6.3 Core/Inc/app.h File Reference

"Clase" [App](#): contiene TODA la lógica del programa (maquina de estados del super-loop, orquestación de [AHT20Sensor](#), Timer y [SerialDisplay](#)). Es el único modulo del TP que [main.c](#) conoce.

```
#include "stm32f4xx_hal.h"
```

Typedefs

- typedef struct App [App](#)

Instancia del programa. Es opaca para el resto del sistema: [main.c](#) solo declara una variable de este tipo y la pasa por puntero a [App_Init\(\)](#)/[App_Run\(\)](#); nunca toca sus campos.

Functions

- [App * App_GetInstance](#) (void)

Devuelve un puntero a la (única) instancia estatica de [App](#). Evita que [main.c](#) tenga que conocer el tamaño de la estructura (que queda totalmente privada en [app.c](#)).

- void [App_Init](#) ([App](#) *self, I2C_HandleTypeDef *hi2c, UART_HandleTypeDef *huart)

Inicializa el programa: configura [SerialDisplay](#) y [AHT20Sensor](#) sobre los handles de HAL ya inicializados por [main\(\)](#), y deja el sistema listo para arrancar el super-loop.

- void [App_Run](#) ([App](#) *self)

Un tick del super-loop de la aplicación. Se llama en cada vuelta del while(1) de [main\(\)](#); internamente avanza la maquina de estados (INIT/WAITING/MEASURING/DISPLAYING/ERROR) sin bloquear salvo en MEASURING (ver informe, seccion de modelado de comportamiento).

6.3.1 Detailed Description

"Clase" [App](#): contiene TODA la lógica del programa (maquina de estados del super-loop, orquestación de [AHT20Sensor](#), Timer y [SerialDisplay](#)). Es el único modulo del TP que [main.c](#) conoce.

Diseño (RNF-02, SRP): [main.c](#) no sabe nada de I2C, UART, ni del protocolo del AHT20 – solo inicializa la HAL y le entrega los handles a [App_Init\(\)](#). Todo lo demas (que hacer con esos handles, cuando leer, que imprimir) vive acá adentro.

Definition in file [app.h](#).

6.3.2 Typedef Documentation

6.3.2.1 App

```
typedef struct App App
```

Instancia del programa. Es opaca para el resto del sistema: [main.c](#) solo declara una variable de este tipo y la pasa por puntero a [App_Init\(\)/App_Run\(\)](#); nunca toca sus campos.

Definition at line 28 of file [app.h](#).

6.3.3 Function Documentation

6.3.3.1 App_GetInstance()

```
App * App_GetInstance (
    void )
```

Devuelve un puntero a la (única) instancia estática de [App](#). Evita que [main.c](#) tenga que conocer el tamaño de la estructura (que queda totalmente privada en [app.c](#)).

Definition at line 54 of file [app.c](#).

```
00055 {
00056     return &s_app;
00057 }
```

References [App_GetInstance\(\)](#), and [s_app](#).

6.3.3.2 App_Init()

```
void App_Init (
    App * self,
    I2C_HandleTypeDef * hi2c,
    UART_HandleTypeDef * huart)
```

Inicializa el programa: configura [SerialDisplay](#) y [AHT20Sensor](#) sobre los handles de HAL ya inicializados por [main\(\)](#), y deja el sistema listo para arrancar el super-loop.

Parameters

<i>self</i>	instancia devuelta por App_GetInstance()
<i>hi2c</i>	handle de I2C ya inicializado (MX_I2C1_Init)
<i>huart</i>	handle de UART ya inicializado (MX_USART2_UART_Init)

Definition at line 59 of file [app.c](#).

```
00060 {
00061     SerialDisplay_Init(&self->display_, huart);
00062     self->state_ = APP_STATE_INIT;
00063
00064     if (AHT20_Init(&self->sensor_, hi2c)) {
00065         self->state_ = APP_STATE_WAITING;
00066         self->lastReadTime_ = Timer_Millis();
00067     } else {
00068         self->state_ = APP_STATE_ERROR;
00069         self->errorTime_ = Timer_Millis();
00070         SerialDisplay_PrintError(&self->display_, "AHT20 no responde en el bus I2C");
00071     }
00072 }
```

References [AHT20_Init\(\)](#), [App_Init\(\)](#), [APP_STATE_ERROR](#), [APP_STATE_INIT](#), [APP_STATE_WAITING](#), [App::display_](#), [App::errorTime_](#), [App::lastReadTime_](#), [App::sensor_](#), [SerialDisplay_Init\(\)](#), [SerialDisplay_PrintError\(\)](#), [App::state_](#), and [Timer_Millis\(\)](#).

6.3.3.3 App_Run()

```
void App_Run (
    App * self)
```

Un tick del super-loop de la aplicación. Se llama en cada vuelta del while(1) de [main\(\)](#); internamente avanza la maquina de estados (INIT/WAITING/MEASURING/DISPLAYING/ERROR) sin bloquear salvo en MEASURING (ver informe, seccion de modelado de comportamiento).

Definition at line 74 of file [app.c](#).

```
00075 {
00076     switch (self->state_) {
00077     case APP_STATE_WAITING:
00078         if ((Timer_Millis() - self->lastReadTime_) >= APP_SAMPLE_PERIOD_MS) {
00079             self->state_ = APP_STATE_MEASURING;
00080         }
00081         break;
00082     case APP_STATE_MEASURING:
00083         if (AHT20_ReadMeasurement(&self->sensor_)) {
00084             self->state_ = APP_STATE_DISPLAYING;
00085         } else {
00086             self->state_ = APP_STATE_ERROR;
00087         }
00088         break;
00089     case APP_STATE_DISPLAYING:
00090         SerialDisplay_PrintMeasurement(&self->display_,
00091                                     AHT20_GetTemperature(&self->sensor_),
00092                                     AHT20_GetHumidity(&self->sensor_));
00093         self->lastReadTime_ = Timer_Millis();
00094         self->state_ = APP_STATE_WAITING;
00095         break;
00096     case APP_STATE_ERROR:
00097         if (Timer_Millis() - self->errorTime_ >= 2000) {
00098             SerialDisplay_PrintError(&self->display_, "Lectura AHT20 fallida, presione RESET para
00099 reiniciar");
00100             self->lastReadTime_ = Timer_Millis();
00101             self->state_ = APP_STATE_INIT;
00102         }
00103         break;
00104     case APP_STATE_INIT:
00105         break;
00106     default:
00107         /* No deberia llegar aca: App_Init() siempre deja WAITING o ERROR */
00108         self->state_ = APP_STATE_ERROR;
00109         break;
00110     }
00111 }
```

References [AHT20_GetHumidity\(\)](#), [AHT20_GetTemperature\(\)](#), [AHT20_ReadMeasurement\(\)](#), [App_Run\(\)](#), [APP_SAMPLE_PERIOD_MS](#), [APP_STATE_DISPLAYING](#), [APP_STATE_ERROR](#), [APP_STATE_INIT](#), [APP_STATE_MEASURING](#), [APP_STATE_WAITING](#), [App::display_](#), [App::errorTime_](#), [App::lastReadTime_](#), [App::sensor_](#), [SerialDisplay_PrintError\(\)](#), [SerialDisplay_PrintMeasurement\(\)](#), [App::state_](#), and [Timer_Millis\(\)](#).

6.4 app.h

[Go to the documentation of this file.](#)

```
00001
00014 #ifndef APP_H
00015 #define APP_H
00016
00017 #ifdef __cplusplus
00018 extern "C" {
00019 #endif
00020
00021 #include "stm32f4xx_hal.h"
00022
```

```
00028 typedef struct App App;
00029
00035 App *App_GetInstance(void);
00036
00045 void App_Init(App *self, I2C_HandleTypeDef *hi2c, UART_HandleTypeDef *huart);
00046
00053 void App_Run(App *self);
00054
00055 #ifdef __cplusplus
00056 }
00057 #endif
00058
00059 #endif /* APP_H */
```

6.5 Core/Inc/main.h File Reference

: Header for [main.c](#) file. This file contains the common defines of the application.

```
#include "stm32f4xx_hal.h"
```

Functions

- void [Error_Handler](#) (void)
This function is executed in case of error occurrence.

6.5.1 Detailed Description

: Header for [main.c](#) file. This file contains the common defines of the application.

Attention

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

Definition in file [main.h](#).

6.5.2 Function Documentation

6.5.2.1 Error_Handler()

```
void Error_Handler (
    void )
```

This function is executed in case of error occurrence.

Return values

<i>None</i>	
-------------	--

Definition at line 250 of file [main.c](#).

```
00251 {
00252     /* USER CODE BEGIN Error_Handler_Debug */
00253     /* User can add his own implementation to report the HAL error return state */
00254     __disable_irq();
00255     while (1)
00256     {
00257     }
00258     /* USER CODE END Error_Handler_Debug */
00259 }
```

References [Error_Handler\(\)](#).

6.6 main.h

[Go to the documentation of this file.](#)

```
00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020
00021 /* Define to prevent recursive inclusion -----*/
00022 #ifndef __MAIN_H
00023 #define __MAIN_H
00024
00025 #ifdef __cplusplus
00026 extern "C" {
00027 #endif
00028
00029 /* Includes -----*/
00030 #include "stm32f4xx_hal.h"
00031
00032 /* Private includes -----*/
00033 /* USER CODE BEGIN Includes */
00034
00035 /* USER CODE END Includes */
00036
00037 /* Exported types -----*/
00038 /* USER CODE BEGIN ET */
00039
00040 /* USER CODE END ET */
00041
00042 /* Exported constants -----*/
00043 /* USER CODE BEGIN EC */
00044
00045 /* USER CODE END EC */
00046
00047 /* Exported macro -----*/
00048 /* USER CODE BEGIN EM */
00049
00050 /* USER CODE END EM */
00051
00052 /* Exported functions prototypes -----*/
00053 void Error\_Handler(void);
00054
00055 /* USER CODE BEGIN EFP */
00056
00057 /* USER CODE END EFP */
00058
00059 /* Private defines -----*/
00060
00061 /* USER CODE BEGIN Private defines */
00062
00063 /* USER CODE END Private defines */
00064
00065 #ifdef __cplusplus
00066 }
00067 #endif
00068
00069 #endif /* __MAIN_H */
```

6.7 Core/Inc/serial_display.h File Reference

"Clase" [SerialDisplay](#): envía texto por el UART2 (puerto virtual del ST-LINK) para mostrar la temperatura en una terminal PuTTY, a 115200 8N1.

```
#include "stm32f4xx_hal.h"
```

Classes

- struct [SerialDisplay](#)

Functions

- void [SerialDisplay_Init](#) ([SerialDisplay](#) *self, UART_HandleTypeDef *huart)
Asocia el objeto al periférico UART ya configurado por CubeMX.
- void [SerialDisplay_PrintMeasurement](#) ([SerialDisplay](#) *self, float temperature_c, float humidity_rh)
Imprime una línea con la temperatura y humedad actuales.
- void [SerialDisplay_PrintError](#) ([SerialDisplay](#) *self, const char *message)
Imprime un mensaje de error (p. ej. fallo de lectura I2C).

6.7.1 Detailed Description

"Clase" [SerialDisplay](#): envía texto por el UART2 (puerto virtual del ST-LINK) para mostrar la temperatura en una terminal PuTTY, a 115200 8N1.

Definition in file [serial_display.h](#).

6.7.2 Function Documentation

6.7.2.1 SerialDisplay_Init()

```
void SerialDisplay_Init (
    SerialDisplay * self,
    UART_HandleTypeDef * huart)
```

Asocia el objeto al periférico UART ya configurado por CubeMX.

Inicializa la interfaz de salida serie.

Asocia el objeto al periférico UART ya configurado por CubeMX.

Asocia la instancia de [SerialDisplay](#) con el periférico UART proporcionado.

Parameters

in, out	self	Puntero a la instancia de SerialDisplay .
---------	------	---

<i>in</i>	<i>huart</i>	Puntero al manejador del periférico UART utilizado para la transmisión.
-----------	--------------	---

Definition at line 32 of file [serial_display.c](#).

```
00033 {
00034     self->huart_ = huart;
00035 }
```

References [SerialDisplay::huart_](#), and [SerialDisplay_Init\(\)](#).

6.7.2.2 SerialDisplay_PrintError()

```
void SerialDisplay_PrintError (
    SerialDisplay * self,
    const char * message)
```

Imprime un mensaje de error (p. ej. fallo de lectura I2C).

Transmite un mensaje de error por UART.

Imprime un mensaje de error (p. ej. fallo de lectura I2C).

Agrega el prefijo "[ERROR]" al mensaje recibido y lo transmite mediante el periférico UART asociado a la instancia.

Formato de salida:

```
[ERROR] mensaje
```

Parameters

<i>in</i>	<i>self</i>	Puntero a la instancia de SerialDisplay .
<i>in</i>	<i>message</i>	Puntero al mensaje de error terminado en '\0'.

Definition at line 78 of file [serial_display.c](#).

```
00079 {
00080     char line[64];
00081     int len = snprintf(line, sizeof(line), "[ERROR] %s\r\n", message);
00082
00083     if (len > 0) {
00084         HAL_UART_Transmit(self->huart_, (uint8_t *)line, (uint16_t)len, SERIAL_TX_TIMEOUT_MS);
00085     }
00086 }
```

References [SerialDisplay::huart_](#), [SERIAL_TX_TIMEOUT_MS](#), and [SerialDisplay_PrintError\(\)](#).

6.7.2.3 SerialDisplay_PrintMeasurement()

```
void SerialDisplay_PrintMeasurement (
    SerialDisplay * self,
    float temperature_c,
    float humidity_rh)
```

Imprime una línea con la temperatura y humedad actuales.

Transmite una medición de temperatura y humedad por UART.

Imprime una línea con la temperatura y humedad actuales.

Formatea los valores recibidos y los transmite mediante el periférico UART asociado a la instancia. La temperatura y la humedad se muestran con una precisión de un decimal.

Formato de salida:

Temp: XX.X C Hum: XX.X %RH

Parameters

in	<i>self</i>	Puntero a la instancia de SerialDisplay .
in	<i>temperature_c</i>	Temperatura en grados Celsius.
in	<i>humidity_rh</i>	Humedad relativa en porcentaje.

Definition at line 53 of file [serial_display.c](#).

```

00054 {
00055     char line[64];
00056     int len = snprintf(line, sizeof(line), "Temp: %.1f C Hum: %.1f %%RH\r\n",
00057                         temperature_c, humidity_rh);
00058
00059     if (len > 0) {
00060         HAL_UART_Transmit(self->huart_, (uint8_t *)line, (uint16_t)len, SERIAL_TX_TIMEOUT_MS);
00061     }
00062 }
```

References [SerialDisplay::huart_](#), [SERIAL_TX_TIMEOUT_MS](#), and [SerialDisplay_PrintMeasurement\(\)](#).

6.8 serial_display.h

[Go to the documentation of this file.](#)

```

00001
00010
00011 #ifndef SERIAL_DISPLAY_H
00012 #define SERIAL_DISPLAY_H
00013
00014 #ifdef __cplusplus
00015 extern "C" {
00016 #endif
00017
00018 #include "stm32f4xx_hal.h"
00019
00020 typedef struct {
00021     UART_HandleTypeDef *huart_;
00022 } SerialDisplay;
00023
00025 void SerialDisplay_Init(SerialDisplay *self, UART_HandleTypeDef *huart);
00026
00028 void SerialDisplay_PrintMeasurement(SerialDisplay *self, float temperature_c, float humidity_rh);
00029
00031 void SerialDisplay_PrintError(SerialDisplay *self, const char *message);
00032
00033 #ifdef __cplusplus
00034 }
00035 #endif
00036
00037 #endif /* SERIAL_DISPLAY_H */
```

6.9 Core/Inc/timer.h File Reference

Modulo "utility" con métodos estáticos para medir tiempo sin bloquear el super-loop (RNF-01) y sin acoplar el resto del software a una implementacion concreta de reloj (RNF-03).

```
#include <stdint.h>
```

Functions

- uint32_t [Timer_Millis](#) (void)
Milisegundos transcurridos desde el arranque del sistema. Hoy delega en HAL_GetTick(); si se migra de HAL a otra capa (o a otro microcontrolador), solo se reescribe [timer.c](#).
- void [Timer_Delay](#) (uint32_t ms)
Espera bloqueante en milisegundos (solo para init/medicion).

6.9.1 Detailed Description

Modulo "utility" con métodos estáticos para medir tiempo sin bloquear el super-loop (RNF-01) y sin acoplar el resto del software a una implementacion concreta de reloj (RNF-03).

Definition in file [timer.h](#).

6.9.2 Function Documentation

6.9.2.1 Timer_Delay()

```
void Timer_Delay (  
    uint32_t ms)
```

Espera bloqueante en milisegundos (solo para init/medicion).

Genera un retardo bloqueante.

Espera bloqueante en milisegundos (solo para init/medicion).

Suspende la ejecución del programa durante la cantidad de milisegundos especificada utilizando la función de retardo proporcionada por la HAL de STM32.

Parameters

in	ms	Tiempo de retardo en milisegundos.
----	----	------------------------------------

Definition at line 37 of file [timer.c](#).

```
00038 {  
00039     HAL_Delay (ms);  
00040 }
```

References [Timer_Delay\(\)](#).

6.9.2.2 Timer_Millis()

```
uint32_t Timer_Millis (
    void )
```

Milisegundos transcurridos desde el arranque del sistema. Hoy delega en HAL_GetTick(); si se migra de HAL a otra capa (o a otro microcontrolador), solo se reescribe [timer.c](#).

Obtiene el tiempo transcurrido desde el inicio del sistema.

Milisegundos transcurridos desde el arranque del sistema. Hoy delega en HAL_GetTick(); si se migra de HAL a otra capa (o a otro microcontrolador), solo se reescribe [timer.c](#).

Retorna el valor del contador de tiempo del sistema proporcionado por la HAL de STM32. El valor se expresa en milisegundos y se incrementa periódicamente mediante la interrupción del sistema.

Returns

Tiempo transcurrido desde el inicio del sistema, expresado en milisegundos.

Definition at line 23 of file [timer.c](#).

```
00024 {
00025     return HAL_GetTick();
00026 }
```

References [Timer_Millis\(\)](#).

6.10 timer.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef TIMER_H
00010 #define TIMER_H
00011
00012 #ifdef __cplusplus
00013 extern "C" {
00014 #endif
00015
00016 #include <stdint.h>
00017
00023 uint32_t Timer_Millis(void);
00024
00026 void Timer_Delay(uint32_t ms);
00027
00028 #ifdef __cplusplus
00029 }
00030 #endif
00031
00032 #endif /* TIMER_H */
```

6.11 Core/Src/aht20.c File Reference

Implementación del driver AHT20 (I2C).

```
#include "aht20.h"
```


Macros

- `#define AHT20_CMD_INIT {0xBE, 0x08, 0x00}`
- `#define AHT20_CMD_MEASURE {0xAC, 0x33, 0x00}`
- `#define AHT20_STATUS_BUSY_MASK 0x80U`
- `#define AHT20_STATUS_CAL_MASK 0x08U`
- `#define AHT20_I2C_TIMEOUT_MS 100U`

Functions

- static bool `AHT20_ReadStatus` (`AHT20Sensor *self`, `uint8_t *status`)
- bool `AHT20_Init` (`AHT20Sensor *self`, `I2C_HandleTypeDef *hi2c`)
Inicializa el objeto sensor y verifica/calibra el AHT20.
- bool `AHT20_ReadMeasurement` (`AHT20Sensor *self`)
Dispara una medición y actualiza temperature_/humidity_. Es una operación semi-bloqueante (~80 ms) por el tiempo de conversión propio del sensor; se llama desde el estado MEASURING del super-loop, no dentro de un ISR.
- float `AHT20_GetTemperature` (`const AHT20Sensor *self`)
Última temperatura valida leida, en grados Celsius.
- float `AHT20_GetHumidity` (`const AHT20Sensor *self`)
Última humedad relativa valida leida, en RH.

6.11.1 Detailed Description

Implementación del driver AHT20 (I2C).

Protocolo del sensor (hoja de datos AHT20): 1) Al energizar, esperar ≥ 40 ms. 2) Verificar bit de calibración (bit 3) del byte de estado; si no esta seteado, enviar comando de inicializacion 0xBE 0x08 0x00. 3) Para medir: enviar 0xAC 0x33 0x00 y esperar ≥ 80 ms. 4) Leer 6 (o 7) bytes: [status][RH 20 bits][T 20 bits (comparten 1 byte)].

Definition in file [aht20.c](#).

6.11.2 Macro Definition Documentation

6.11.2.1 AHT20_CMD_INIT

```
#define AHT20_CMD_INIT {0xBE, 0x08, 0x00}
```

Definition at line 17 of file [aht20.c](#).

6.11.2.2 AHT20_CMD_MEASURE

```
#define AHT20_CMD_MEASURE {0xAC, 0x33, 0x00}
```

Definition at line 18 of file [aht20.c](#).

6.11.2.3 AHT20_I2C_TIMEOUT_MS

```
#define AHT20_I2C_TIMEOUT_MS 100U
```

Definition at line 21 of file [aht20.c](#).

6.11.2.4 AHT20_STATUS_BUSY_MASK

```
#define AHT20_STATUS_BUSY_MASK 0x80U
```

Definition at line 19 of file [aht20.c](#).

6.11.2.5 AHT20_STATUS_CAL_MASK

```
#define AHT20_STATUS_CAL_MASK 0x08U
```

Definition at line 20 of file [aht20.c](#).

6.11.3 Function Documentation

6.11.3.1 AHT20_GetHumidity()

```
float AHT20_GetHumidity (  
    const AHT20Sensor * self)
```

Última humedad relativa valida leida, en RH.

Definition at line 100 of file [aht20.c](#).

```
00101 {  
00102     return self->humidity_  
00103 }
```

References [AHT20_GetHumidity\(\)](#), and [AHT20Sensor::humidity_](#).

6.11.3.2 AHT20_GetTemperature()

```
float AHT20_GetTemperature (  
    const AHT20Sensor * self)
```

Última temperatura valida leida, en grados Celsius.

Definition at line 95 of file [aht20.c](#).

```
00096 {  
00097     return self->temperature_  
00098 }
```

References [AHT20_GetTemperature\(\)](#), and [AHT20Sensor::temperature_](#).

6.11.3.3 AHT20_Init()

```
bool AHT20_Init (
    AHT20Sensor * self,
    I2C_HandleTypeDef * hi2c)
```

Inicializa el objeto sensor y verifica/calibra el AHT20.

Parameters

<i>self</i>	instancia del sensor (ya con hi2c_ asignado por el caller)
-------------	--

Return values

<i>true</i>	si el sensor respondió y quedó calibrado, false si no.
-------------	--

Definition at line 26 of file [aht20.c](#).

```
00027 {
00028     uint8_t init_cmd[3] = AHT20_CMD_INIT;
00029     uint8_t status = 0;
00030
00031     self->hi2c_ = hi2c;
00032     self->address_ = AHT20_I2C_ADDRESS;
00033     self->temperature_ = 0.0f;
00034     self->humidity_ = 0.0f;
00035     self->calibrated_ = false;
00036
00037     HAL_Delay(40); /* tiempo minimo de power-on del AHT20 */
00038
00039     if (!AHT20_ReadStatus(self, &status)) {
00040         return false;
00041     }
00042
00043     if ((status & AHT20_STATUS_CAL_MASK) == 0U) {
00044         if (HAL_I2C_Master_Transmit(self->hi2c_, self->address_, init_cmd,
00045                                     sizeof(init_cmd), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00046             return false;
00047         }
00048         HAL_Delay(10);
00049     }
00050
00051     if (!AHT20_ReadStatus(self, &status)) {
00052         return false;
00053     }
00054
00055     self->calibrated_ = ((status & AHT20_STATUS_CAL_MASK) != 0U);
00056     return self->calibrated_;
00057 }
```

References [AHT20Sensor::address_](#), [AHT20_CMD_INIT](#), [AHT20_I2C_ADDRESS](#), [AHT20_I2C_TIMEOUT_MS](#), [AHT20_Init\(\)](#), [AHT20_ReadStatus\(\)](#), [AHT20_STATUS_CAL_MASK](#), [AHT20Sensor::calibrated_](#), [AHT20Sensor::hi2c_](#), [AHT20Sensor::humidity_](#), [status](#), and [AHT20Sensor::temperature_](#).

6.11.3.4 AHT20_ReadMeasurement()

```
bool AHT20_ReadMeasurement (
    AHT20Sensor * self)
```

Dispara una medición y actualiza temperature_/humidity_. Es una operación semi-bloqueante (~80 ms) por el tiempo de conversión propio del sensor; se llama desde el estado MEASURING del super-loop, no dentro de un ISR.

Return values

<code>true</code>	si la lectura fue valida (ACK + CRC de estado ok).
-------------------	--

Definition at line 59 of file [aht20.c](#).

```

00060 {
00061     uint8_t measure_cmd[3] = AHT20_CMD_MEASURE;
00062     uint8_t data[6] = {0};
00063     uint32_t raw_hum;
00064     uint32_t raw_temp;
00065
00066     if (!self->calibrated_) {
00067         return false;
00068     }
00069
00070     if (HAL_I2C_Master_Transmit(self->hi2c_, self->address_, measure_cmd,
00071                                sizeof(measure_cmd), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00072         return false;
00073     }
00074
00075     HAL_Delay(80); /* tiempo de conversion tipico del AHT20 */
00076
00077     if (HAL_I2C_Master_Receive(self->hi2c_, self->address_, data,
00078                                sizeof(data), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00079         return false;
00080     }
00081
00082     if ((data[0] & AHT20_STATUS_BUSY_MASK) != 0U) {
00083         return false; /* el sensor seguia ocupado: descartar lectura */
00084     }
00085
00086     raw_hum = ((uint32_t)data[1] << 12) | ((uint32_t)data[2] << 4) | (data[3] >> 4);
00087     raw_temp = (((uint32_t)data[3] & 0x0FU) << 16) | ((uint32_t)data[4] << 8) | data[5];
00088
00089     self->humidity_ = ((float)raw_hum / 1048576.0f) * 100.0f;
00090     self->temperature_ = (((float)raw_temp / 1048576.0f) * 200.0f) - 50.0f;
00091
00092     return true;
00093 }

```

References [AHT20Sensor::address_](#), [AHT20_CMD_MEASURE](#), [AHT20_I2C_TIMEOUT_MS](#), [AHT20_ReadMeasurement\(\)](#), [AHT20_STATUS_BUSY_MASK](#), [AHT20Sensor::calibrated_](#), [AHT20Sensor::hi2c_](#), [AHT20Sensor::humidity_](#), and [AHT20Sensor::temperature_](#).

6.11.3.5 AHT20_ReadStatus()

```

bool AHT20_ReadStatus (
    AHT20Sensor * self,
    uint8_t * status) [static]

```

Definition at line 106 of file [aht20.c](#).

```

00107 {
00108     return HAL_I2C_Master_Receive(self->hi2c_, self->address_, status, 1,
00109                                   AHT20_I2C_TIMEOUT_MS) == HAL_OK;
00110 }

```

References [AHT20Sensor::address_](#), [AHT20_I2C_TIMEOUT_MS](#), [AHT20_ReadStatus\(\)](#), [AHT20Sensor::hi2c_](#), and [status](#).

6.12 aht20.c

[Go to the documentation of this file.](#)

```

00001
00014 #include "aht20.h"
00015
00016 /* ---- Constantes privadas del modulo (no expuestas en el .h) ----- */

```

```

00017 #define AHT20_CMD_INIT          {0xBE, 0x08, 0x00}
00018 #define AHT20_CMD_MEASURE        {0xAC, 0x33, 0x00}
00019 #define AHT20_STATUS_BUSY_MASK   0x80U
00020 #define AHT20_STATUS_CAL_MASK    0x08U
00021 #define AHT20_I2C_TIMEOUT_MS     100U
00022
00023 /* ---- Prototipos privados ----- */
00024 static bool AHT20_ReadStatus(AHT20Sensor *self, uint8_t *status);
00025
00026 bool AHT20_Init(AHT20Sensor *self, I2C_HandleTypeDef *hi2c)
00027 {
00028     uint8_t init_cmd[3] = AHT20_CMD_INIT;
00029     uint8_t status = 0;
00030
00031     self->hi2c_ = hi2c;
00032     self->address_ = AHT20_I2C_ADDRESS;
00033     self->temperature_ = 0.0f;
00034     self->humidity_ = 0.0f;
00035     self->calibrated_ = false;
00036
00037     HAL_Delay(40); /* tiempo minimo de power-on del AHT20 */
00038
00039     if (!AHT20_ReadStatus(self, &status)) {
00040         return false;
00041     }
00042
00043     if ((status & AHT20_STATUS_CAL_MASK) == 0U) {
00044         if (HAL_I2C_Master_Transmit(self->hi2c_, self->address_, init_cmd,
00045                                     sizeof(init_cmd), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00046             return false;
00047         }
00048         HAL_Delay(10);
00049     }
00050
00051     if (!AHT20_ReadStatus(self, &status)) {
00052         return false;
00053     }
00054
00055     self->calibrated_ = ((status & AHT20_STATUS_CAL_MASK) != 0U);
00056     return self->calibrated_;
00057 }
00058
00059 bool AHT20_ReadMeasurement(AHT20Sensor *self)
00060 {
00061     uint8_t measure_cmd[3] = AHT20_CMD_MEASURE;
00062     uint8_t data[6] = {0};
00063     uint32_t raw_hum;
00064     uint32_t raw_temp;
00065
00066     if (!self->calibrated_) {
00067         return false;
00068     }
00069
00070     if (HAL_I2C_Master_Transmit(self->hi2c_, self->address_, measure_cmd,
00071                                 sizeof(measure_cmd), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00072         return false;
00073     }
00074
00075     HAL_Delay(80); /* tiempo de conversion tipico del AHT20 */
00076
00077     if (HAL_I2C_Master_Receive(self->hi2c_, self->address_, data,
00078                                sizeof(data), AHT20_I2C_TIMEOUT_MS) != HAL_OK) {
00079         return false;
00080     }
00081
00082     if ((data[0] & AHT20_STATUS_BUSY_MASK) != 0U) {
00083         return false; /* el sensor seguia ocupado: descartar lectura */
00084     }
00085
00086     raw_hum = ((uint32_t)data[1] << 12) | ((uint32_t)data[2] << 4) | (data[3] >> 4);
00087     raw_temp = ((uint32_t)data[3] & 0x0FU) << 16 | ((uint32_t)data[4] << 8) | data[5];
00088
00089     self->humidity_ = ((float)raw_hum / 1048576.0f) * 100.0f;
00090     self->temperature_ = (((float)raw_temp / 1048576.0f) * 200.0f) - 50.0f;
00091
00092     return true;
00093 }
00094
00095 float AHT20_GetTemperature(const AHT20Sensor *self)
00096 {
00097     return self->temperature_;
00098 }
00099
00100 float AHT20_GetHumidity(const AHT20Sensor *self)
00101 {
00102     return self->humidity_;
00103 }

```

```

00104
00105 /* ---- Privados ----- */
00106 static bool AHT20_ReadStatus(AHT20Sensor *self, uint8_t *status)
00107 {
00108     return HAL_I2C_Master_Receive(self->hi2c_, self->address_, status, 1,
00109                                   AHT20_I2C_TIMEOUT_MS) == HAL_OK;
00110 }

```

6.13 Core/Src/app.c File Reference

Implementación de la clase [App](#). Aca vive toda la lógica del programa: la definición real de la estructura (privada a este archivo, siguiendo el patron de "puntero a estructura anonima" visto en clase 7 para representar objetos en C) y la máquina de estados del super-loop.

```

#include "app.h"
#include "aht20.h"
#include "timer.h"
#include "serial_display.h"
#include "String.h"

```

Classes

- struct [App](#)

Definición real de [App](#). Es privada a [app.c](#): fuera de este archivo solo se conoce como "struct App" (tipo incompleto), por lo que [main.c](#) no puede acceder a estos campos ni le importa como estan organizados (encapsulamiento).

Macros

- #define [APP_SAMPLE_PERIOD_MS](#) 5000U

Enumerations

- enum [AppState](#) {
[APP_STATE_INIT](#) = 0 , [APP_STATE_WAITING](#) , [APP_STATE_MEASURING](#) , [APP_STATE_DISPLAYING](#) ,
[APP_STATE_ERROR](#) }

Estados del ciclo de medición (ver diagrama de estados del TP). Corresponden al patron "Observar y reaccionar" con Super Loop no bloqueante (RNF-01): WAITING solo consulta [Timer_Millis\(\)](#), nunca usa [HAL_Delay\(\)](#) en ese estado.

Functions

- [App * App_GetInstance](#) (void)
Devuelve un puntero a la (única) instancia estatica de [App](#). Evita que [main.c](#) tenga que conocer el tamaño de la estructura (que queda totalmente privada en [app.c](#)).
- void [App_Init](#) ([App](#) *self, I2C_HandleTypeDef *hi2c, UART_HandleTypeDef *huart)
Inicializa el programa: configura [SerialDisplay](#) y [AHT20Sensor](#) sobre los handles de HAL ya inicializados por [main\(\)](#), y deja el sistema listo para arrancar el super-loop.
- void [App_Run](#) ([App](#) *self)
Un tick del super-loop de la aplicación. Se llama en cada vuelta del while(1) de [main\(\)](#); internamente avanza la máquina de estados (INIT/WAITING/MEASURING/DISPLAYING/ERROR) sin bloquear salvo en MEASURING (ver informe, seccion de modelado de comportamiento).

Variables

- HAL_StatusTypeDef [status](#)
- uint32_t [error](#)
- static [App s_app](#)

6.13.1 Detailed Description

Implementación de la clase [App](#). Aca vive toda la lógica del programa: la definición real de la estructura (privada a este archivo, siguiendo el patron de "puntero a estructura anonima" visto en clase 7 para representar objetos en C) y la máquina de estados del super-loop.

Definition in file [app.c](#).

6.13.2 Macro Definition Documentation

6.13.2.1 APP_SAMPLE_PERIOD_MS

```
#define APP_SAMPLE_PERIOD_MS 5000U
```

Definition at line [49](#) of file [app.c](#).

6.13.3 Enumeration Type Documentation

6.13.3.1 AppState

```
enum AppState
```

Estados del ciclo de medición (ver diagrama de estados del TP). Corresponden al patron "Observar y reaccionar" con Super Loop no bloqueante (RNF-01): WAITING solo consulta [Timer_Millis\(\)](#), nunca usa HAL_Delay() en ese estado.

Enumerator

APP_STATE_INIT	
APP_STATE_WAITING	
APP_STATE_MEASURING	
APP_STATE_DISPLAYING	
APP_STATE_ERROR	

Definition at line [24](#) of file [app.c](#).

```
00024     {
00025         APP_STATE_INIT = 0,
00026         APP_STATE_WAITING,
00027         APP_STATE_MEASURING,
00028         APP_STATE_DISPLAYING,
00029         APP_STATE_ERROR
00030     } AppState;
```

6.13.4 Function Documentation

6.13.4.1 App_GetInstance()

```
App * App_GetInstance (
    void )
```

Devuelve un puntero a la (única) instancia estática de [App](#). Evita que [main.c](#) tenga que conocer el tamaño de la estructura (que queda totalmente privada en [app.c](#)).

Definition at line 54 of file [app.c](#).

```
00055 {
00056     return &s_app;
00057 }
```

References [App_GetInstance\(\)](#), and [s_app](#).

6.13.4.2 App_Init()

```
void App_Init (
    App * self,
    I2C_HandleTypeDef * hi2c,
    UART_HandleTypeDef * huart)
```

Inicializa el programa: configura [SerialDisplay](#) y [AHT20Sensor](#) sobre los handles de HAL ya inicializados por [main\(\)](#), y deja el sistema listo para arrancar el super-loop.

Parameters

<i>self</i>	instancia devuelta por App_GetInstance()
<i>hi2c</i>	handle de I2C ya inicializado (MX_I2C1_Init)
<i>huart</i>	handle de UART ya inicializado (MX_USART2_UART_Init)

Definition at line 59 of file [app.c](#).

```
00060 {
00061     SerialDisplay_Init(&self->display_, huart);
00062     self->state_ = APP_STATE_INIT;
00063
00064     if (AHT20_Init(&self->sensor_, hi2c)) {
00065         self->state_ = APP_STATE_WAITING;
00066         self->lastReadTime_ = Timer_Millis();
00067     } else {
00068         self->state_ = APP_STATE_ERROR;
00069         self->errorTime_ = Timer_Millis();
00070         SerialDisplay_PrintError(&self->display_, "AHT20 no responde en el bus I2C");
00071     }
00072 }
```

References [AHT20_Init\(\)](#), [App_Init\(\)](#), [APP_STATE_ERROR](#), [APP_STATE_INIT](#), [APP_STATE_WAITING](#), [App::display_](#), [App::errorTime_](#), [App::lastReadTime_](#), [App::sensor_](#), [SerialDisplay_Init\(\)](#), [SerialDisplay_PrintError\(\)](#), [App::state_](#), and [Timer_Millis\(\)](#).

6.13.4.3 App_Run()

```
void App_Run (
    App * self)
```

Un tick del super-loop de la aplicación. Se llama en cada vuelta del while(1) de [main\(\)](#); internamente avanza la maquina de estados (INIT/WAITING/MEASURING/DISPLAYING/ERROR) sin bloquear salvo en MEASURING (ver informe, seccion de modelado de comportamiento).

Definition at line 74 of file [app.c](#).

```
00075 {
00076     switch (self->state_) {
00077
00078     case APP_STATE_WAITING:
00079         if ((Timer_Millis() - self->lastReadTime_) >= APP_SAMPLE_PERIOD_MS) {
00080             self->state_ = APP_STATE_MEASURING;
00081         }
00082         break;
00083
00084     case APP_STATE_MEASURING:
00085         if (AHT20_ReadMeasurement(&self->sensor_)) {
00086             self->state_ = APP_STATE_DISPLAYING;
00087         } else {
00088             self->state_ = APP_STATE_ERROR;
00089         }
00090         break;
00091
00092     case APP_STATE_DISPLAYING:
00093         SerialDisplay_PrintMeasurement(&self->display_,
00094                                     AHT20_GetTemperature(&self->sensor_),
00095                                     AHT20_GetHumidity(&self->sensor_));
00096         self->lastReadTime_ = Timer_Millis();
00097         self->state_ = APP_STATE_WAITING;
00098         break;
00099
00100     case APP_STATE_ERROR:
00101         if (Timer_Millis() - self->errorTime_ >= 2000) {
00102             SerialDisplay_PrintError(&self->display_, "Lectura AHT20 fallida, presione RESET para
reiniciar");
00103             self->lastReadTime_ = Timer_Millis();
00104             self->state_ = APP_STATE_INIT;
00105         }
00106         break;
00107
00108     case APP_STATE_INIT:
00109         break;
00110
00111     default:
00112         /* No deberia llegar aca: App_Init() siempre deja WAITING o ERROR */
00113         self->state_ = APP_STATE_ERROR;
00114         break;
00115     }
00116 }
```

References [AHT20_GetHumidity\(\)](#), [AHT20_GetTemperature\(\)](#), [AHT20_ReadMeasurement\(\)](#), [App_Run\(\)](#), [APP_SAMPLE_PERIOD_MS](#), [APP_STATE_DISPLAYING](#), [APP_STATE_ERROR](#), [APP_STATE_INIT](#), [APP_STATE_MEASURING](#), [APP_STATE_WAITING](#), [App::display_](#), [App::errorTime_](#), [App::lastReadTime_](#), [App::sensor_](#), [SerialDisplay_PrintError\(\)](#), [SerialDisplay_PrintMeasurement\(\)](#), [App::state_](#), and [Timer_Millis\(\)](#).

6.13.5 Variable Documentation

6.13.5.1 error

```
uint32_t error
```

Definition at line 47 of file [app.c](#).

6.13.5.2 s_app

`App s_app [static]`

Definition at line 52 of file [app.c](#).

6.13.5.3 status

`HAL_StatusTypeDef status`

Definition at line 46 of file [app.c](#).

6.14 app.c

[Go to the documentation of this file.](#)

```

00001
00011 #include "app.h"
00012 #include "aht20.h"
00013 #include "timer.h"
00014 #include "serial_display.h"
00015
00016 #include "String.h"
00017
00024 typedef enum {
00025     APP_STATE_INIT = 0,
00026     APP_STATE_WAITING,
00027     APP_STATE_MEASURING,
00028     APP_STATE_DISPLAYING,
00029     APP_STATE_ERROR
00030 } AppState;
00031
00038 struct App {
00039     AHT20Sensor    sensor_;
00040     SerialDisplay  display_;
00041     AppState        state_;
00042     uint32_t        lastReadTime_;
00043     uint32_t        errorTime_;
00044 };
00045
00046 HAL_StatusTypeDef status;
00047 uint32_t error;
00048
00049 #define APP_SAMPLE_PERIOD_MS 5000U /* RF-01/RF-02: periodo de muestreo */
00050
00051 /* Única instancia del programa (RNF-05: sin heap, todo estatico). */
00052 static App s_app;
00053
00054 App *App_GetInstance(void)
00055 {
00056     return &s_app;
00057 }
00058
00059 void App_Init(App *self, I2C_HandleTypeDef *hi2c, UART_HandleTypeDef *huart)
00060 {
00061     SerialDisplay_Init(&self->display_, huart);
00062     self->state_ = APP_STATE_INIT;
00063
00064     if (AHT20_Init(&self->sensor_, hi2c)) {
00065         self->state_ = APP_STATE_WAITING;
00066         self->lastReadTime_ = Timer_Millis();
00067     } else {
00068         self->state_ = APP_STATE_ERROR;
00069         self->errorTime_ = Timer_Millis();
00070         SerialDisplay_PrintError(&self->display_, "AHT20 no responde en el bus I2C");
00071     }
00072 }
00073
00074 void App_Run(App *self)
00075 {
00076     switch (self->state_) {
00077     case APP_STATE_WAITING:

```

```

00079         if ((Timer_Millis() - self->lastReadTime_) >= APP_SAMPLE_PERIOD_MS) {
00080             self->state_ = APP_STATE_MEASURING;
00081         }
00082         break;
00083     case APP_STATE_MEASURING:
00084         if (AHT20_ReadMeasurement(&self->sensor_)) {
00085             self->state_ = APP_STATE_DISPLAYING;
00086         } else {
00087             self->state_ = APP_STATE_ERROR;
00088         }
00089         break;
00090     case APP_STATE_DISPLAYING:
00091         SerialDisplay_PrintMeasurement(&self->display_,
00092                                     AHT20_GetTemperature(&self->sensor_),
00093                                     AHT20_GetHumidity(&self->sensor_));
00094         self->lastReadTime_ = Timer_Millis();
00095         self->state_ = APP_STATE_WAITING;
00096         break;
00097     case APP_STATE_ERROR:
00098         if (Timer_Millis() - self->errorTime_ >= 2000) {
00099             SerialDisplay_PrintError(&self->display_, "Lectura AHT20 fallida, presione RESET para
00100 reiniciar");
00101             self->lastReadTime_ = Timer_Millis();
00102             self->state_ = APP_STATE_INIT;
00103         }
00104         break;
00105     case APP_STATE_INIT:
00106         break;
00107     default:
00108         /* No deberia llegar aca: App_Init() siempre deja WAITING o ERROR */
00109         self->state_ = APP_STATE_ERROR;
00110         break;
00111     }
00112 }
00113 }

```

6.15 Core/Src/main.c File Reference

: Main program body

```

#include "main.h"
#include "app.h"

```

Functions

- void [SystemClock_Config](#) (void)
System Clock Configuration.
- static void [MX_GPIO_Init](#) (void)
GPIO Initialization Function.
- static void [MX_USART2_UART_Init](#) (void)
USART2 Initialization Function.
- static void [MX_I2C1_Init](#) (void)
I2C1 Initialization Function.
- int [main](#) (void)
The application entry point.
- void [Error_Handler](#) (void)
This function is executed in case of error occurrence.

Variables

- I2C_HandleTypeDef [hi2c1](#)
- UART_HandleTypeDef [huart2](#)

6.15.1 Detailed Description

: Main program body

Attention

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

Definition in file [main.c](#).

6.15.2 Function Documentation

6.15.2.1 Error_Handler()

```
void Error_Handler (
    void )
```

This function is executed in case of error occurrence.

Return values

<i>None</i>	
-------------	--

Definition at line 250 of file [main.c](#).

```
00251 {
00252     /* USER CODE BEGIN Error_Handler_Debug */
00253     /* User can add his own implementation to report the HAL error return state */
00254     __disable_irq();
00255     while (1)
00256     {
00257     }
00258     /* USER CODE END Error_Handler_Debug */
00259 }
```

References [Error_Handler\(\)](#).

6.15.2.2 main()

```
int main (
    void )
```

The application entry point.

Return values

<i>int</i>	
------------	--

Definition at line 70 of file [main.c](#).

```
00071 {
00072
00073     /* USER CODE BEGIN 1 */
00074
00075     /* USER CODE END 1 */
00076
00077     /* MCU Configuration-----*/
00078
00079     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
00080     HAL_Init();
00081
00082     /* USER CODE BEGIN Init */
00083
00084     /* USER CODE END Init */
00085
00086     /* Configure the system clock */
00087     SystemClock_Config();
00088
00089     /* USER CODE BEGIN SysInit */
00090
00091     /* USER CODE END SysInit */
00092
00093     /* Initialize all configured peripherals */
00094     MX_GPIO_Init();
00095     MX_USART2_UART_Init();
00096     MX_I2C1_Init();
00097     /* USER CODE BEGIN 2 */
00098     App *app = App_GetInstance();
00099     App_Init(app, &hi2c1, &huart2);
00100     /* USER CODE END 2 */
00101
00102     /* Infinite loop */
00103     /* USER CODE BEGIN WHILE */
00104     while (1)
00105     {
00106         App_Run(app);
00107         /* USER CODE END WHILE */
00108
00109         /* USER CODE BEGIN 3 */
00110     }
00111     /* USER CODE END 3 */
00112 }
```

References [App_GetInstance\(\)](#), [App_Init\(\)](#), [App_Run\(\)](#), [hi2c1](#), [huart2](#), [main\(\)](#), [MX_GPIO_Init\(\)](#), [MX_I2C1_Init\(\)](#), [MX_USART2_UART_Init\(\)](#), and [SystemClock_Config\(\)](#).

6.15.2.3 MX_GPIO_Init()

```
void MX_GPIO_Init (
    void ) [static]
```

GPIO Initialization Function.

Parameters

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

Definition at line 227 of file [main.c](#).

```

00228 {
00229     /* USER CODE BEGIN MX_GPIO_Init_1 */
00230
00231     /* USER CODE END MX_GPIO_Init_1 */
00232
00233     /* GPIO Ports Clock Enable */
00234     __HAL_RCC_GPIOA_CLK_ENABLE();
00235     __HAL_RCC_GPIOB_CLK_ENABLE();
00236
00237     /* USER CODE BEGIN MX_GPIO_Init_2 */
00238
00239     /* USER CODE END MX_GPIO_Init_2 */
00240 }
```

References [MX_GPIO_Init\(\)](#).

6.15.2.4 MX_I2C1_Init()

```

void MX_I2C1_Init (
    void ) [static]
```

I2C1 Initialization Function.

Parameters

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

Definition at line 160 of file [main.c](#).

```

00161 {
00162
00163     /* USER CODE BEGIN I2C1_Init 0 */
00164
00165     /* USER CODE END I2C1_Init 0 */
00166
00167     /* USER CODE BEGIN I2C1_Init 1 */
00168
00169     /* USER CODE END I2C1_Init 1 */
00170     hi2c1.Instance = I2C1;
00171     hi2c1.Init.ClockSpeed = 100000;
00172     hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
00173     hi2c1.Init.OwnAddress1 = 0;
00174     hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
00175     hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
00176     hi2c1.Init.OwnAddress2 = 0;
00177     hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
00178     hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
00179     if (HAL_I2C_Init(&hi2c1) != HAL_OK)
00180     {
00181         Error_Handler();
00182     }
00183     /* USER CODE BEGIN I2C1_Init 2 */
00184
00185     /* USER CODE END I2C1_Init 2 */
00186
00187 }
```

References [Error_Handler\(\)](#), [hi2c1](#), and [MX_I2C1_Init\(\)](#).

6.15.2.5 MX_USART2_UART_Init()

```
void MX_USART2_UART_Init (
    void ) [static]
```

USART2 Initialization Function.

Parameters

None	
------	--

Return values

None	
------	--

Definition at line 194 of file [main.c](#).

```
00195 {
00196
00197     /* USER CODE BEGIN USART2_Init 0 */
00198
00199     /* USER CODE END USART2_Init 0 */
00200
00201     /* USER CODE BEGIN USART2_Init 1 */
00202
00203     /* USER CODE END USART2_Init 1 */
00204     huart2.Instance = USART2;
00205     huart2.Init.BaudRate = 115200;
00206     huart2.Init.WordLength = UART_WORDLENGTH_8B;
00207     huart2.Init.StopBits = UART_STOPBITS_1;
00208     huart2.Init.Parity = UART_PARITY_NONE;
00209     huart2.Init.Mode = UART_MODE_TX_RX;
00210     huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
00211     huart2.Init.OverSampling = UART_OVERSAMPLING_16;
00212     if (HAL_UART_Init(&huart2) != HAL_OK)
00213     {
00214         Error_Handler();
00215     }
00216     /* USER CODE BEGIN USART2_Init 2 */
00217
00218     /* USER CODE END USART2_Init 2 */
00219
00220 }
```

References [Error_Handler\(\)](#), [huart2](#), and [MX_USART2_UART_Init\(\)](#).

6.15.2.6 SystemClock_Config()

```
void SystemClock_Config (
    void )
```

System Clock Configuration.

Return values

None	
------	--

Configure the main internal regulator output voltage

Initializes the RCC Oscillators according to the specified parameters in the RCC_OscInitTypeDef structure.

Initializes the CPU, AHB and APB buses clocks

Definition at line 118 of file [main.c](#).

```

00119 {
00120     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
00121     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
00122
00125     __HAL_RCC_PWR_CLK_ENABLE();
00126     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
00127
00131     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
00132     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
00133     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
00134     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
00135     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
00136     {
00137         Error_Handler();
00138     }
00139
00142     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSClk
00143                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
00144     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
00145     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
00146     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
00147     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
00148
00149     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
00150     {
00151         Error_Handler();
00152     }
00153 }
```

References [Error_Handler\(\)](#), and [SystemClock_Config\(\)](#).

6.15.3 Variable Documentation

6.15.3.1 hi2c1

I2C_HandleTypeDef hi2c1

Definition at line 44 of file [main.c](#).

6.15.3.2 huart2

UART_HandleTypeDef huart2

Definition at line 46 of file [main.c](#).

6.16 main.c

[Go to the documentation of this file.](#)

```

00001 /* USER CODE BEGIN Header */
00018 /* USER CODE END Header */
00019 /* Includes -----*/
00020 #include "main.h"
00021
00022 /* Private includes -----*/
00023 /* USER CODE BEGIN Includes */
00024 #include "app.h"
00025 /* USER CODE END Includes */
00026
00027 /* Private typedef -----*/
00028 /* USER CODE BEGIN PTD */
00029
00030
```



```

00031 /* USER CODE END PTD */
00032
00033 /* Private define -----*/
00034 /* USER CODE BEGIN PD */
00035
00036 /* USER CODE END PD */
00037
00038 /* Private macro -----*/
00039 /* USER CODE BEGIN PM */
00040
00041 /* USER CODE END PM */
00042
00043 /* Private variables -----*/
00044 I2C_HandleTypeDef hi2c1;
00045
00046 UART_HandleTypeDef huart2;
00047
00048 /* USER CODE BEGIN PV */
00049
00050 /* USER CODE END PV */
00051
00052 /* Private function prototypes -----*/
00053 void SystemClock_Config(void);
00054 static void MX_GPIO_Init(void);
00055 static void MX_USART2_UART_Init(void);
00056 static void MX_I2C1_Init(void);
00057 /* USER CODE BEGIN PFP */
00058
00059 /* USER CODE END PFP */
00060
00061 /* Private user code -----*/
00062 /* USER CODE BEGIN 0 */
00063
00064 /* USER CODE END 0 */
00065
00070 int main(void)
00071 {
00072
00073     /* USER CODE BEGIN 1 */
00074
00075     /* USER CODE END 1 */
00076
00077     /* MCU Configuration-----*/
00078
00079     /* Reset of all peripherals, Initializes the Flash interface and the SysTick. */
00080     HAL_Init();
00081
00082     /* USER CODE BEGIN Init */
00083
00084     /* USER CODE END Init */
00085
00086     /* Configure the system clock */
00087     SystemClock_Config();
00088
00089     /* USER CODE BEGIN SysInit */
00090
00091     /* USER CODE END SysInit */
00092
00093     /* Initialize all configured peripherals */
00094     MX_GPIO_Init();
00095     MX_USART2_UART_Init();
00096     MX_I2C1_Init();
00097     /* USER CODE BEGIN 2 */
00098     App *app = App_GetInstance();
00099     App_Init(app, &hi2c1, &huart2);
00100     /* USER CODE END 2 */
00101
00102     /* Infinite loop */
00103     /* USER CODE BEGIN WHILE */
00104     while (1)
00105     {
00106         App_Run(app);
00107         /* USER CODE END WHILE */
00108
00109         /* USER CODE BEGIN 3 */
00110     }
00111     /* USER CODE END 3 */
00112 }
00113
00118 void SystemClock_Config(void)
00119 {
00120     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
00121     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
00122
00123     __HAL_RCC_PWR_CLK_ENABLE();
00124     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
00125
00126
00127

```

```

00131 RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
00132 RCC_OscInitStruct.HSISState = RCC_HSI_ON;
00133 RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
00134 RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
00135 if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
00136 {
00137     Error_Handler();
00138 }
00139
00142 RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
00143                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
00144 RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
00145 RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
00146 RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
00147 RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
00148
00149 if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
00150 {
00151     Error_Handler();
00152 }
00153 }
00154
00160 static void MX_I2C1_Init(void)
00161 {
00162
00163     /* USER CODE BEGIN I2C1_Init 0 */
00164
00165     /* USER CODE END I2C1_Init 0 */
00166
00167     /* USER CODE BEGIN I2C1_Init 1 */
00168
00169     /* USER CODE END I2C1_Init 1 */
00170     hi2c1.Instance = I2C1;
00171     hi2c1.Init.ClockSpeed = 100000;
00172     hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
00173     hi2c1.Init.OwnAddress1 = 0;
00174     hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
00175     hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
00176     hi2c1.Init.OwnAddress2 = 0;
00177     hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
00178     hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
00179     if (HAL_I2C_Init(&hi2c1) != HAL_OK)
00180     {
00181         Error_Handler();
00182     }
00183     /* USER CODE BEGIN I2C1_Init 2 */
00184
00185     /* USER CODE END I2C1_Init 2 */
00186
00187 }
00188
00194 static void MX_USART2_UART_Init(void)
00195 {
00196
00197     /* USER CODE BEGIN USART2_Init 0 */
00198
00199     /* USER CODE END USART2_Init 0 */
00200
00201     /* USER CODE BEGIN USART2_Init 1 */
00202
00203     /* USER CODE END USART2_Init 1 */
00204     huart2.Instance = USART2;
00205     huart2.Init.BaudRate = 115200;
00206     huart2.Init.WordLength = UART_WORDLENGTH_8B;
00207     huart2.Init.StopBits = UART_STOPBITS_1;
00208     huart2.Init.Parity = UART_PARITY_NONE;
00209     huart2.Init.Mode = UART_MODE_TX_RX;
00210     huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
00211     huart2.Init.OverSampling = UART_OVERSAMPLING_16;
00212     if (HAL_UART_Init(&huart2) != HAL_OK)
00213     {
00214         Error_Handler();
00215     }
00216     /* USER CODE BEGIN USART2_Init 2 */
00217
00218     /* USER CODE END USART2_Init 2 */
00219
00220 }
00221
00227 static void MX_GPIO_Init(void)
00228 {
00229     /* USER CODE BEGIN MX_GPIO_Init_1 */
00230
00231     /* USER CODE END MX_GPIO_Init_1 */
00232
00233     /* GPIO Ports Clock Enable */
00234     __HAL_RCC_GPIOA_CLK_ENABLE();

```

```

00235  __HAL_RCC_GPIOB_CLK_ENABLE();
00236
00237  /* USER CODE BEGIN MX_GPIO_Init_2 */
00238
00239  /* USER CODE END MX_GPIO_Init_2 */
00240 }
00241
00242 /* USER CODE BEGIN 4 */
00243
00244 /* USER CODE END 4 */
00245
00250 void Error_Handler(void)
00251 {
00252     /* USER CODE BEGIN Error_Handler_Debug */
00253     /* User can add his own implementation to report the HAL error return state */
00254     __disable_irq();
00255     while (1)
00256     {
00257     }
00258     /* USER CODE END Error_Handler_Debug */
00259 }
00260 #ifdef USE_FULL_ASSERT
00268 void assert_failed(uint8_t *file, uint32_t line)
00269 {
00270     /* USER CODE BEGIN 6 */
00271     /* User can add his own implementation to report the file name and line number,
00272        ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
00273     /* USER CODE END 6 */
00274 }
00275 #endif /* USE_FULL_ASSERT */

```

6.17 Core/Src/serial_display.c File Reference

Implementación de la interfaz de salida por UART para mostrar mediciones y errores.

```

#include "serial_display.h"
#include <stdio.h>
#include <string.h>

```

Macros

- `#define SERIAL_TX_TIMEOUT_MS 100U`
Tiempo máximo de espera para la transmisión UART.

Functions

- void `SerialDisplay_Init` (`SerialDisplay` *self, `UART_HandleTypeDef` *huart)
Inicializa la interfaz de salida serie.
- void `SerialDisplay_PrintMeasurement` (`SerialDisplay` *self, float temperature_c, float humidity_rh)
Transmite una medición de temperatura y humedad por UART.
- void `SerialDisplay_PrintError` (`SerialDisplay` *self, const char *message)
Transmite un mensaje de error por UART.

6.17.1 Detailed Description

Implementación de la interfaz de salida por UART para mostrar mediciones y errores.

Este módulo proporciona funciones para inicializar una interfaz de salida serie y transmitir por UART las mediciones de temperatura y humedad, así como mensajes de error.

Definition in file [serial_display.c](#).

6.17.2 Macro Definition Documentation

6.17.2.1 SERIAL_TX_TIMEOUT_MS

```
#define SERIAL_TX_TIMEOUT_MS 100U
```

Tiempo máximo de espera para la transmisión UART.

Tiempo de espera utilizado por HAL_UART_Transmit() para completar una transmisión.

Definition at line 20 of file [serial_display.c](#).

6.17.3 Function Documentation

6.17.3.1 SerialDisplay_Init()

```
void SerialDisplay_Init (
    SerialDisplay * self,
    UART_HandleTypeDef * huart)
```

Inicializa la interfaz de salida serie.

Asocia el objeto al periférico UART ya configurado por CubeMX.

Asocia la instancia de [SerialDisplay](#) con el periférico UART proporcionado.

Parameters

<i>in, out</i>	<i>self</i>	Puntero a la instancia de SerialDisplay .
<i>in</i>	<i>huart</i>	Puntero al manejador del periférico UART utilizado para la transmisión.

Inicializa la interfaz de salida serie.

Definition at line 32 of file [serial_display.c](#).

```
00033 {
00034     self->huart_ = huart;
00035 }
```

References [SerialDisplay::huart_](#), and [SerialDisplay_Init\(\)](#).

6.17.3.2 SerialDisplay_PrintError()

```
void SerialDisplay_PrintError (
    SerialDisplay * self,
    const char * message)
```

Transmite un mensaje de error por UART.

Imprime un mensaje de error (p. ej. fallo de lectura I2C).

Agrega el prefijo "[ERROR]" al mensaje recibido y lo transmite mediante el periférico UART asociado a la instancia.

Formato de salida:

[ERROR] mensaje

Parameters

in	<i>self</i>	Puntero a la instancia de SerialDisplay .
in	<i>message</i>	Puntero al mensaje de error terminado en '\0'.

Transmite un mensaje de error por UART.

Definition at line 78 of file [serial_display.c](#).

```
00079 {
00080     char line[64];
00081     int len = snprintf(line, sizeof(line), "[ERROR] %s\r\n", message);
00082
00083     if (len > 0) {
00084         HAL_UART_Transmit(self->huart_, (uint8_t *)line, (uint16_t)len, SERIAL_TX_TIMEOUT_MS);
00085     }
00086 }
```

References [SerialDisplay::huart_](#), [SERIAL_TX_TIMEOUT_MS](#), and [SerialDisplay_PrintError\(\)](#).

6.17.3.3 SerialDisplay_PrintMeasurement()

```
void SerialDisplay_PrintMeasurement (
    SerialDisplay * self,
    float temperature_c,
    float humidity_rh)
```

Transmite una medición de temperatura y humedad por UART.

Imprime una línea con la temperatura y humedad actuales.

Formatea los valores recibidos y los transmite mediante el periférico UART asociado a la instancia. La temperatura y la humedad se muestran con una precisión de un decimal.

Formato de salida:

Temp: XX.X C Hum: XX.X %RH

Parameters

in	<i>self</i>	Puntero a la instancia de SerialDisplay .
----	-------------	---

in	<i>temperature_c</i>	Temperatura en grados Celsius.
in	<i>humidity_rh</i>	Humedad relativa en porcentaje.

Transmite una medición de temperatura y humedad por UART.

Definition at line 53 of file [serial_display.c](#).

```

00054 {
00055     char line[64];
00056     int len = snprintf(line, sizeof(line), "Temp: %.1f C   Hum: %.1f %%RH\r\n",
00057                        temperature_c, humidity_rh);
00058
00059     if (len > 0) {
00060         HAL_UART_Transmit(self->huart_, (uint8_t *)line, (uint16_t)len, SERIAL_TX_TIMEOUT_MS);
00061     }
00062 }
```

References [SerialDisplay::huart_](#), [SERIAL_TX_TIMEOUT_MS](#), and [SerialDisplay_PrintMeasurement\(\)](#).

6.18 serial_display.c

[Go to the documentation of this file.](#)

```

00001
00009 #include "serial_display.h"
00010 #include <stdio.h>
00011 #include <string.h>
00012
00013
00020 #define SERIAL_TX_TIMEOUT_MS 100U
00021
00032 void SerialDisplay_Init(SerialDisplay *self, UART_HandleTypeDef *huart)
00033 {
00034     self->huart_ = huart;
00035 }
00036
00053 void SerialDisplay_PrintMeasurement(SerialDisplay *self, float temperature_c, float humidity_rh)
00054 {
00055     char line[64];
00056     int len = snprintf(line, sizeof(line), "Temp: %.1f C   Hum: %.1f %%RH\r\n",
00057                        temperature_c, humidity_rh);
00058
00059     if (len > 0) {
00060         HAL_UART_Transmit(self->huart_, (uint8_t *)line, (uint16_t)len, SERIAL_TX_TIMEOUT_MS);
00061     }
00062 }
00063
00078 void SerialDisplay_PrintError(SerialDisplay *self, const char *message)
00079 {
00080     char line[64];
00081     int len = snprintf(line, sizeof(line), "[ERROR] %s\r\n", message);
00082
00083     if (len > 0) {
00084         HAL_UART_Transmit(self->huart_, (uint8_t *)line, (uint16_t)len, SERIAL_TX_TIMEOUT_MS);
00085     }
00086 }
```

6.19 Core/Src/timer.c File Reference

Implementación de las funciones de temporización del sistema.

```

#include "timer.h"
#include "stm32f4xx_hal.h"
```

Functions

- uint32_t [Timer_Millis](#) (void)
Obtiene el tiempo transcurrido desde el inicio del sistema.
- void [Timer_Delay](#) (uint32_t ms)
Genera un retardo bloqueante.

6.19.1 Detailed Description

Implementación de las funciones de temporización del sistema.

Este módulo proporciona funciones para obtener el tiempo transcurrido desde el inicio del sistema y generar retardos utilizando la implementación de temporización de la HAL de STM32.

Definition in file [timer.c](#).

6.19.2 Function Documentation

6.19.2.1 [Timer_Delay\(\)](#)

```
void Timer_Delay (  
    uint32_t ms)
```

Genera un retardo bloqueante.

Espera bloqueante en milisegundos (solo para init/medicion).

Suspende la ejecución del programa durante la cantidad de milisegundos especificada utilizando la función de retardo proporcionada por la HAL de STM32.

Parameters

in	ms	Tiempo de retardo en milisegundos.
----	----	------------------------------------

Genera un retardo bloqueante.

Definition at line 37 of file [timer.c](#).

```
00038 {  
00039     HAL_Delay(ms);  
00040 }
```

References [Timer_Delay\(\)](#).

6.19.2.2 Timer_Millis()

```
uint32_t Timer_Millis (  
    void )
```

Obtiene el tiempo transcurrido desde el inicio del sistema.

Milisegundos transcurridos desde el arranque del sistema. Hoy delega en HAL_GetTick(); si se migra de HAL a otra capa (o a otro microcontrolador), solo se reescribe [timer.c](#).

Retorna el valor del contador de tiempo del sistema proporcionado por la HAL de STM32. El valor se expresa en milisegundos y se incrementa periódicamente mediante la interrupción del sistema.

Returns

Tiempo transcurrido desde el inicio del sistema, expresado en milisegundos.

Obtiene el tiempo transcurrido desde el inicio del sistema.

Definition at line 23 of file [timer.c](#).

```
00024 {  
00025     return HAL_GetTick();  
00026 }
```

References [Timer_Millis\(\)](#).

6.20 timer.c

[Go to the documentation of this file.](#)

```
00001  
00009 #include "timer.h"  
00010 #include "stm32f4xx_hal.h"  
00011  
00012  
00023 uint32_t Timer_Millis(void)  
00024 {  
00025     return HAL_GetTick();  
00026 }  
00027  
00037 void Timer_Delay(uint32_t ms)  
00038 {  
00039     HAL_Delay(ms);  
00040 }
```